Physics of Data

Jet Clustering

parallel implementation of *Anti-k_T* clustering algorithm

Modern Computing for Physics

September 8, 2026

Giulia Doda

Outline

01

Jet Clustering

the physical problem, the algorithm and the data

02

Algorithm implementation

CPU serial and parallel implementation, GPU parallel implementation

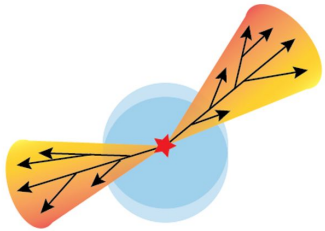
03

Results & Benchmarking

clusters visualization and performance comparison

01

Jet Clustering



QCD processes → Jets

collimated sprays of particles produced by quarks/gluons hadronization flying roughly in the same direction

their reconstruction is essential to understand

01

Anti-k_T algorithm

Jet clustering algorithms

IRC (infrared & collinear) safe

partitional or **hierarchical**/agglomerative

distance measure and **linkage** method

momenta of the final state particles → momenta of a certain number of jets

R: resolution parameter ↔ jet extension

Anti-k_T features

main jet algorithm for LHC collaborations

sequential recombination algorithm

bottom-up approach

clustering based on p_T in the η - ϕ plane

hard jets cluster first

gives **perfectly conical jets**

01

Anti-k_T algorithm

I. distance between particle pairs

$$d_{ij} = \min\left(p_T^{2p}(i), p_T^{2p}(j)\right) \cdot \frac{\Delta R_{ij}^2}{R^2}$$

p=1

$$\Delta R_{ij}^2 = (\eta_i - \eta_j)^2 + (\phi_i - \phi_j)^2$$

II. distance between particles and beam

$$d_{iB} = p_T^{2p}(i)$$

repeat until there are no particles left

III. **merge** (pseudo)particles with the smallest distance **updating** kinematics ***or*** define a **jet** if the distance from beam is the smallest

$$p_T(new) = p_T(i) + p_T(j)$$

$$\eta_{new} = \frac{p_T(i)\eta_i + p_T(j)\eta_j}{p_T(new)}$$

$$\phi_{new} = \frac{p_T(i)\phi_i + p_T(j)\phi_j}{p_T(new)}$$

01

Data

	p_T^0	η^0	ϕ^0	p_T^1	η^1	ϕ^1	p_T^2	η^2	ϕ^2	...
event 0	2.75	-1.23	0.23	9.70	1.13	-2.13	18.43	-0.23	0.97	...
event 1	...	...	...	...	...	...	...	...	...	...
event ...	...	...	...	...	...	...	...	...	...	...

Number of events: 100 000

Max number of particles per event: 700 [zero-padding]

Hierarchical Data Format

02

Serial implementation on CPU

process_event: loop over events

I. read event data and compute number of particles
II. loop over free particles
    A. compute distances between particle pairs
    B. compute distances between particles and beam
    C. find minimum distances
    D. merge
III. save results

02

Serial implementation on CPU

[data handling: structs]

Particle	data type	size (bytes)
particle/pseudocluster ID	int	4
#components	int	4
components ID array	int	4
η, ϕ, p_T, d_B	double	8 x 4 = 32
isJet	bool	1

total: 37 bytes

Event	data type	size (bytes)
event ID	int	4
#particles	int	4
#clusters	int	4
free particles array	int	4
particles array	Particle	37

total: 53 bytes

02

Serial implementation on CPU

[data handling: *filled* structs]

Particle	data type	size (bytes)
particle/pseudocluster ID	int	4
#components	int	4 x 700 = 2800
components ID array	int	4 x 700 = 2800
η, ϕ, p_T, d_B	double	8 x 4 x 700 = 22400
isJet	bool	1

total: ~28 kB

Event	data type	size (bytes)
event ID	int	4
#particles	int	4
#clusters	int	4
free particles array	int	4 x 700 = 2800
particles array	Particle	28 kB x 700 = 19.6 MB

total: ~19.6 MB

02

OpenMP

Parallel implementation on CPU

[same data handling as in serial version]

parallelizing over events
with OpenMP

```
162         #pragma omp parallel for schedule(runtime) // each event per thread
163         for (int ev = 0; ev < N_EVENTS; ++ev) {
164
165             process_single_event(data, ev, times, fout);
166
167         }
```

schedule: clause that controls how loop iterations are split and assigned to threads

and **exploring different schedules**
because each thread workload could be different since each event contains a different #particles

schedule	default chunk size	behavior
static [default]	~ #interations / #threads	iterations are divided into contiguous **block of fixed size** (chunk size), minimum overhead but possible load imbalance
dynamic	1	iterations are divided in blocks but **assigned to blocks runtime**, adaptive to varying workloads, possible overhead
guided	1	**block size decreases over time**, starts with large blocks to minimize scheduling overhead and balances the workload towards the end, good trade off, but not deterministic

02

Parallel implementation on GPU

parallelizing over events: **each event is assigned to a block**

```
processEvent<<<N_blocks, N_thr_bl, shared_mem_size>>>(dev_data, dev_cluster_trace, dev_times);
```

CUDA kernel

I. loop to read the event
II. loop over free particles [each thread handles a particle]
   A. computing distances
   B. **reduction** to find minima [minima are private to each thread]
   C. merging

[different data handling from previous versions]

02

Parallel implementation on GPU

parallelizing over events: **each event is assigned to a block**

```
processEvent<<<N_blocks, N_thr_bl, shared_mem_size>>>(dev_data, dev_cluster_trace, dev_times);
```

clustering results

cluster_trace:
at position *i* we find the particle ID into which particle *i* has been merged

3	129	8	0	...	200
0	*1*	*2*	*3*	*...*	*N*

02

Parallel implementation on GPU

parallelizing over events: **each event is assigned to a block**

```
processEvent<<<N_blocks, N_thr_bl, shared_mem_size>>>(dev_data, dev_cluster_trace, dev_times);
```

shared memory among all threads (limited to block, device code)

```
// shared event data to store kinematics
__shared__ double pt_s[MAX_P];
__shared__ double eta_s[MAX_P];
__shared__ double phi_s[MAX_P];
__shared__ double dB_s[MAX_P];

// to keep track of clusterized particles
__shared__ int free_p_s[MAX_P];
__shared__ int free_particle_counter;
```

dynamic shared memory

host code

```
// calculate amount of dynamic shared memory needed
size_t shared_mem_size = (size_t)N_thr_bl * (2*sizeof(double) + 3*sizeof(int));
```

device code

```
// to handle minima
extern __shared__ char dyn[ ];
double *s_d_ij = (double*) dyn;
int *s_idx_i = (int*)(s_d_ij + block_dim);
int *s_idx_j = (int*)(s_idx_i + block_dim);

double *s_d_iB = (double*)(s_idx_j + block_dim);
int *s_idx_iB = (int*)(s_d_iB + block_dim);
```

02

cloudveneto

Computing details

CPU

```
lscpu
Architecture:                    x86_64
  CPU op-mode(s):                32-bit, 64-bit
  Address sizes:                 46 bits physical, 48 bits virtual
  Byte Order:                    Little Endian
CPU(s):                          15
  On-line CPU(s) list:           0-14
Vendor ID:                       GenuineIntel
  Model name:                    Intel(R) Xeon(R) Gold 5218 CPU @
2.30GHz
    CPU family:                  6
    Model:                       85
    Thread(s) per core:          1
    Core(s) per socket:          1
    Socket(s):                   15
```

GPU

```
GPU name: Tesla T4

Number of 32-bit registers per block: 65536
Number of 32-bit registers per SMP: 65536
Shared memory per block (bytes): 49152 → ~ 48 kB
Shared memory per SMP (bytes): 65536 → ~ 64 kB
Max #threads per block/SMP: 1024
Warp size: 32

#SMPs (multiProcessorCount): 40
CUDA cores (in total): 2560 (since we have 64 CUDA cores per SMP)
Registers (in total): 2621440
Max #threads (all together): 40960

Global memory size (bytes): 15637086208 → ~ 15.6 GB
Constant memory size (bytes): 65536 → ~ 64 kB
L2 cache size (bytes): 4194304 → ~ 4MB

Memory Bus Width (bit): 256
Theoretical BW: 320.06 GB/s
GPU Clock Rate: 1.6 GHz

Block max dim (x,y,z): 1024, 1024, 64
Grid max dim (x,y,z): 2147483647, 65535, 65535

Concurrent kernels: yes
Concurrent computation/communication: yes
```

Clusters Visualization

clustering result file structure (HDF format)

[serial and OpenMP versions]

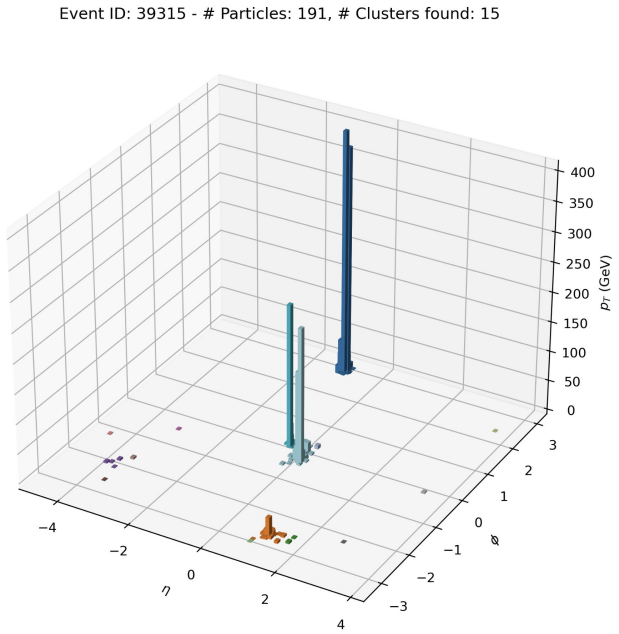
writing results in parallel in OpenMP version

- event *k* group
  - cluster *i* group
    - cluster ID
    - cluster kinematics
    - cluster components (particles IDs)

[CUDA version]

- cluster trace [#events x #particles (max)]
- timings

[serial]

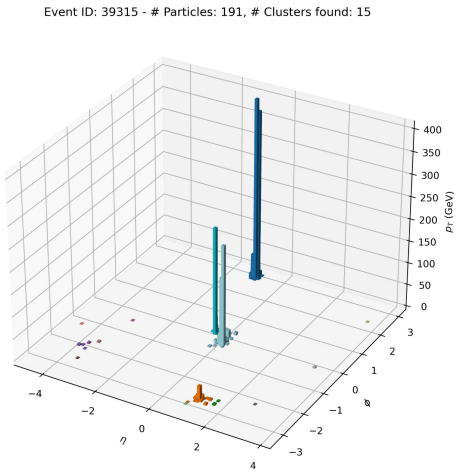


03

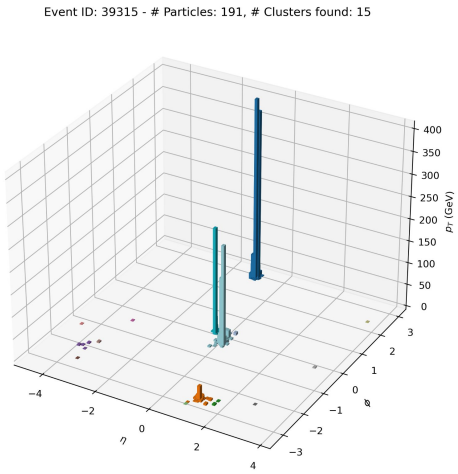
Clusters Visualization

[comparison]

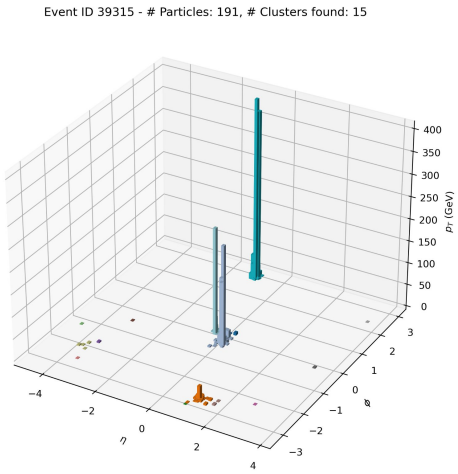
[serial]



[OpenMP]



[CUDA]



same clusters found

03

Benchmarking

[strategy]

Serial

different **compiler optimization levels** and scaling with #events

event execution time VS
#particles in the event

OpenMP

configurations with different
#threads
schedules
schedule chunk size

timing:
whole execution time with
`omp_get_wtime()`

configurations with different
#events
#threads per block

timings:
whole execution time
host to device
kernel
device to host
with `cudaEvent_t`

03

Benchmarking

[metrics]

Speedup

$$S(n) = \frac{T(baseline)}{T(n)}$$

where n = #processing units

relative performance gain compared to an initial (baseline) configuration

ideal: linear

Efficiency

$$E(n) = \frac{S(n)}{n}$$

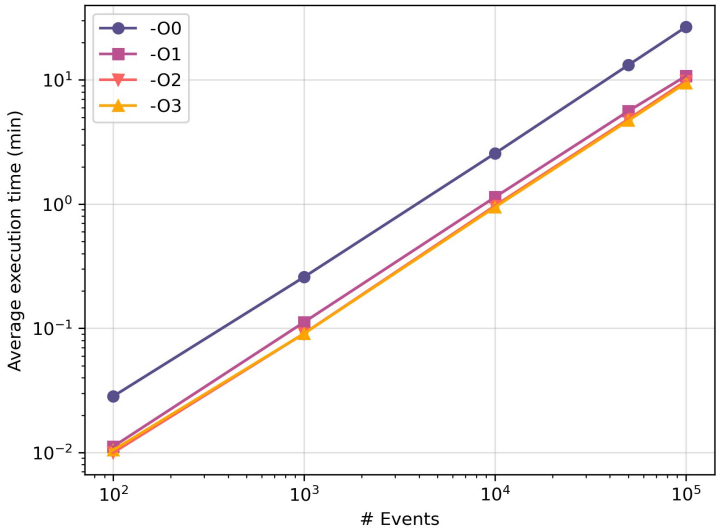
how well extra resources are used

ideal: 1

reality: initialization costs, communication overheads, limited parallel fraction of the process → **Amdahl's law**

Benchmarking

[serial]



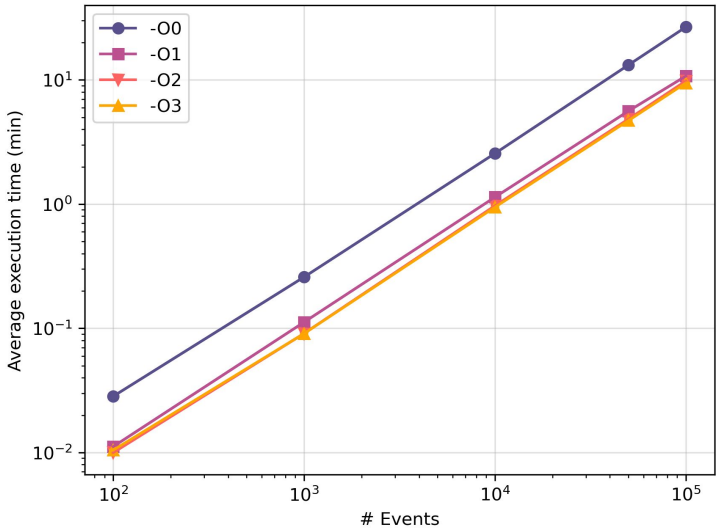
with O1-O3 execution time is less then half the execution time with O0

-O0	no optimization (default)
-O1	minimal optimization, favours compilation time
-O2	intermediate optimization, no speed/size tradeoff
-O3	advanced optimization, favours speed

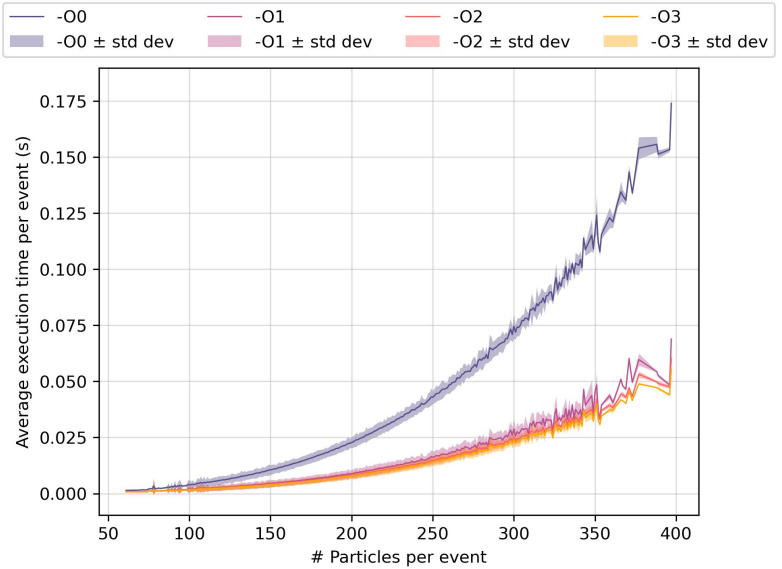
03

Benchmarking

[serial]



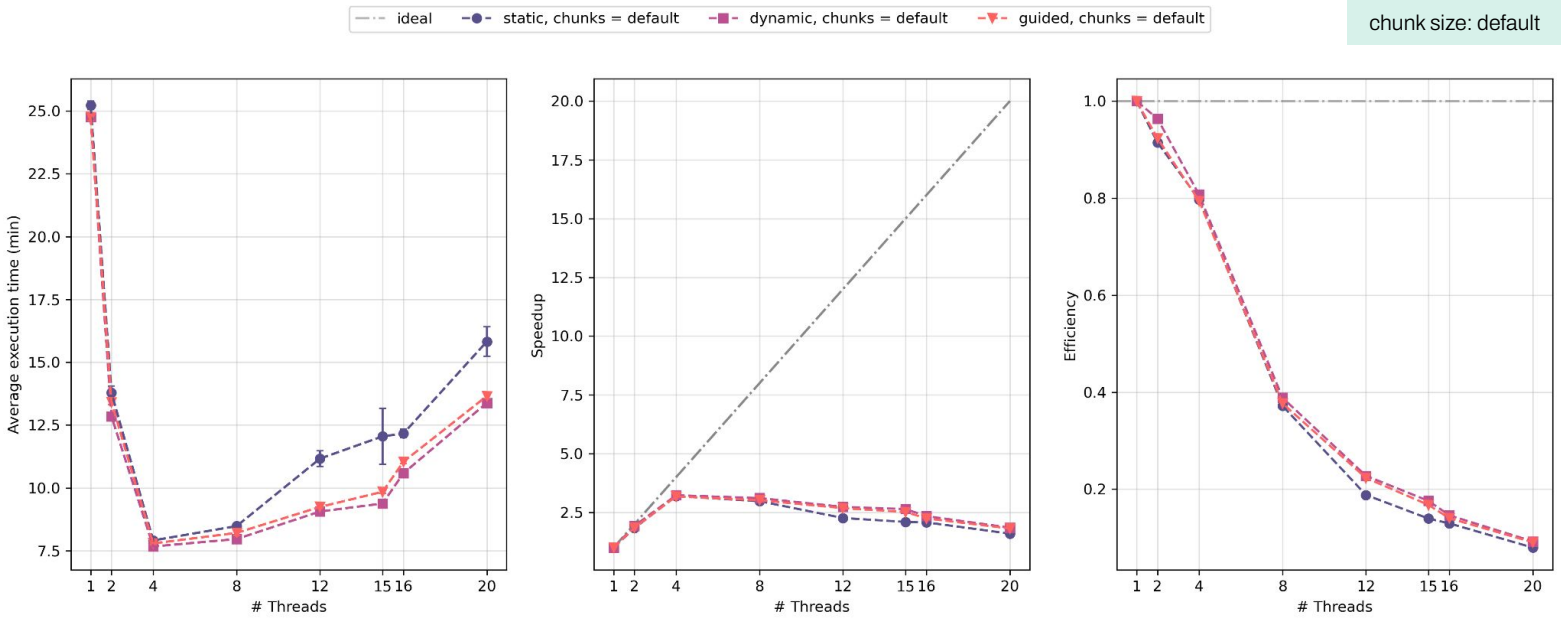
with O1-O3 execution time is less then half the execution time with O0



O1-O3 optimizations offers very similar performances

Benchmarking

OpenMP

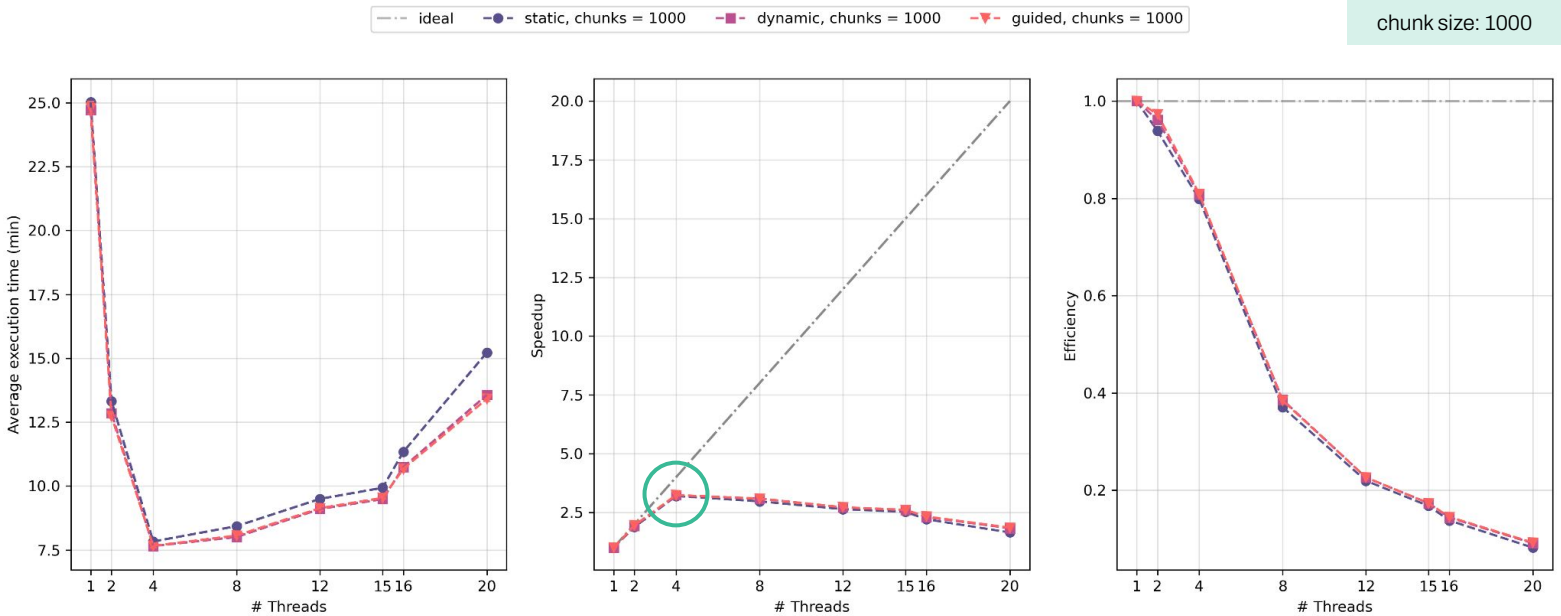


chunk size: default

static schedule is the less stable, efficiency trends follows Amdahl's law

Benchmarking

OpenMP

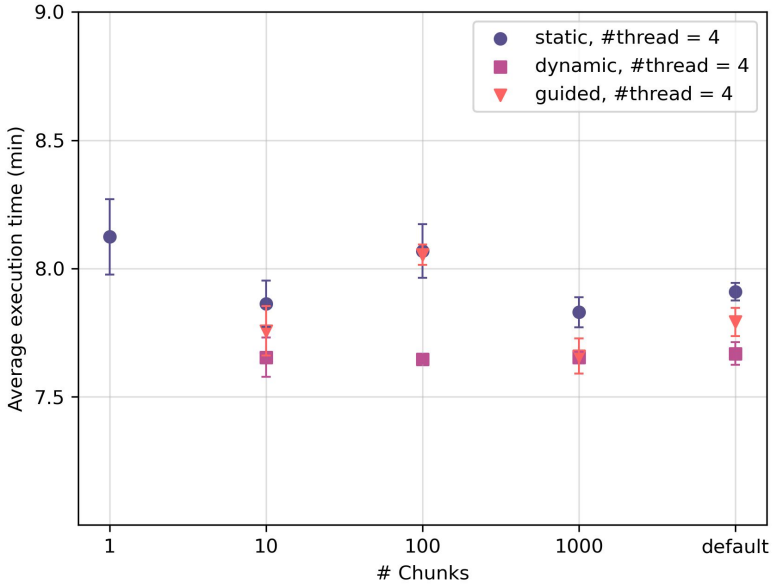


chunk size: 1000

chunk size = 1000 leads to more stability, **4 threads** offer a good trade-off, no difference between schedules

Benchmarking

OpenMP



no big difference between schedules overall

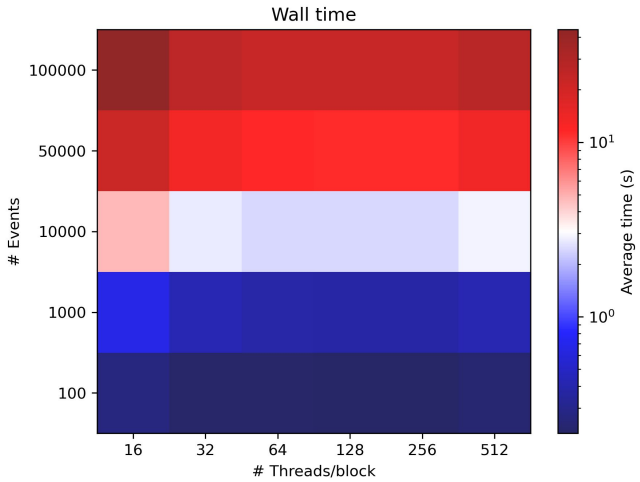
static schedule is the less stable reflecting load imbalance (default chunk size: 25 000)

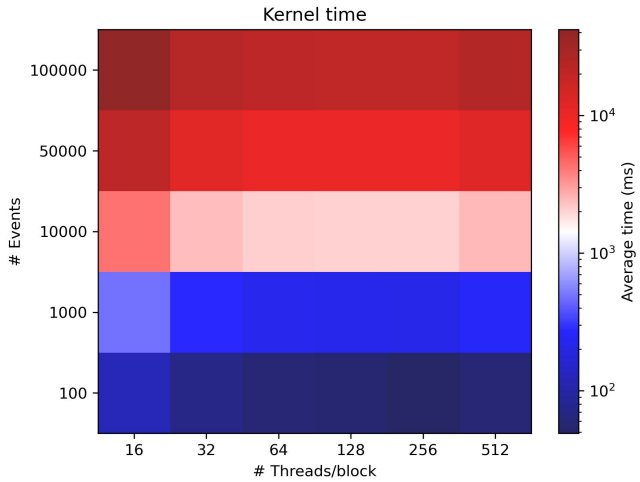
dynamic schedule offers the best trade-off and chunk size does not really matter

guided schedule suffers from instability and chunk size matters

03

Benchmarking

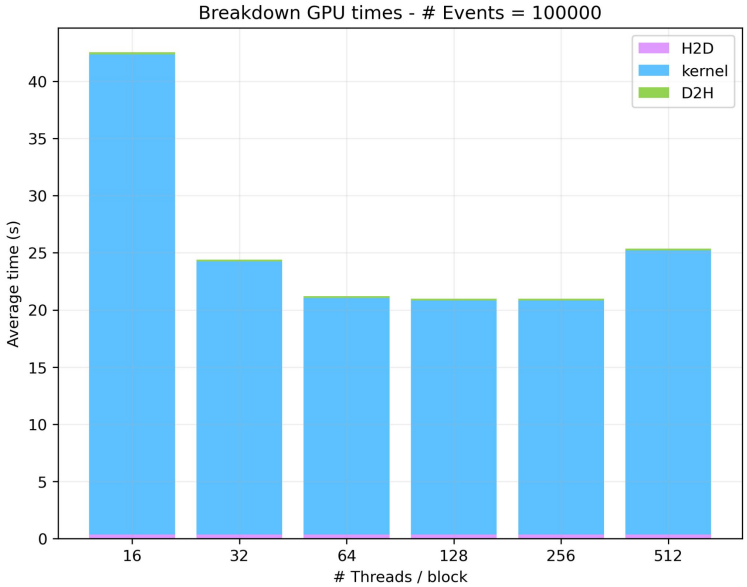




kernel time dominates and it's influenced by the number of particles in each event,
essentially no difference between 64/128/256 threads per block

03

Benchmarking



strongly compute-bound (?)

possible reasons:

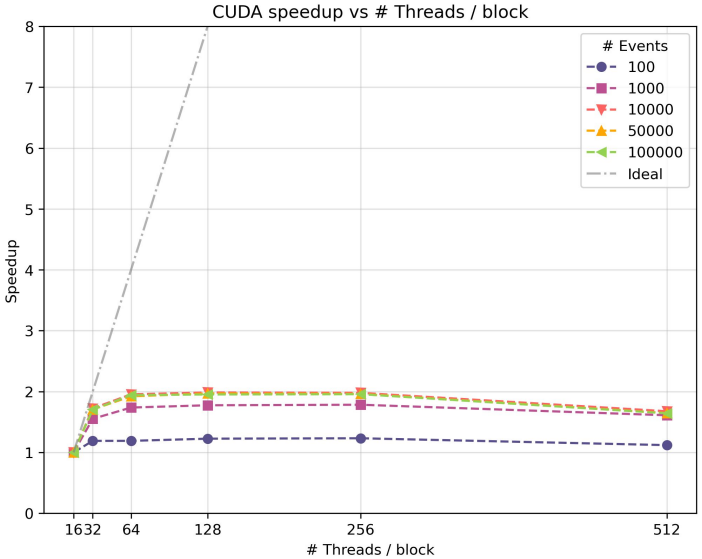
algorithmic complexity

shared memory is reserved for the whole block life, most likely the **bottleneck**

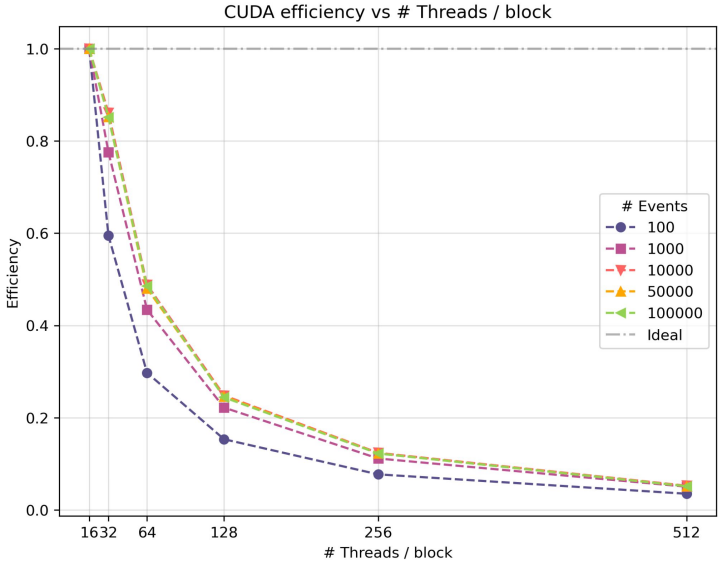
data handling (structure of arrays)

03

Benchmarking



the amount of data clearly matters in exploiting the GPU computational power



the efficiency trend follows Amdahl's law

03

Benchmarking

[metrics]

Bandwidth

$$\text{BW (GB/s)} = \frac{\text{size of transferred memory}}{\text{elapsed kernel time}}$$

amount of data transferred (read + write) per second

to be compared with the theoretical BW

FLOPS

$$\text{FLOPS} = \frac{\text{\#FP operations in a kernel}}{\text{elapsed kernel time}}$$

#FP operations in a kernel per second

for **FP64** ~ 253 GFLOPS [from technical details]

03

Benchmarking

interactive kernel profiler
that provides detailed performance metric

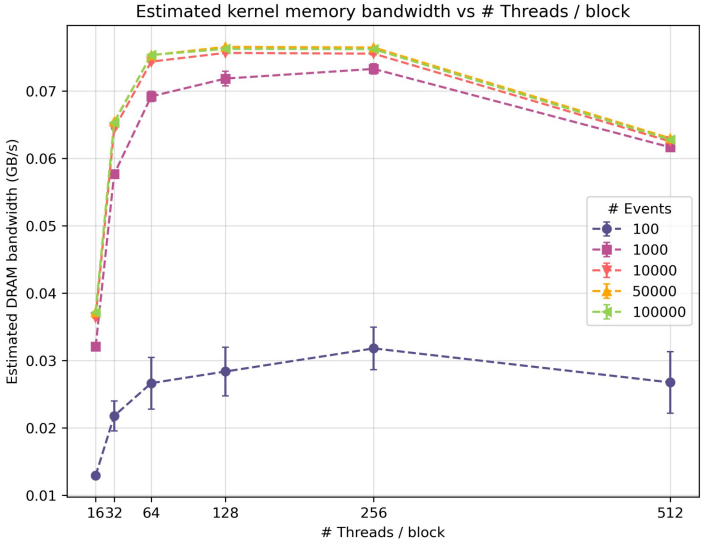
analyzed configuration:
256 threads per block, full dataset

bytes read/written by DRAM	`dram__bytes_read.sum` `dram__bytes_write.sum`	Bytes per event ~ 15.6 MB

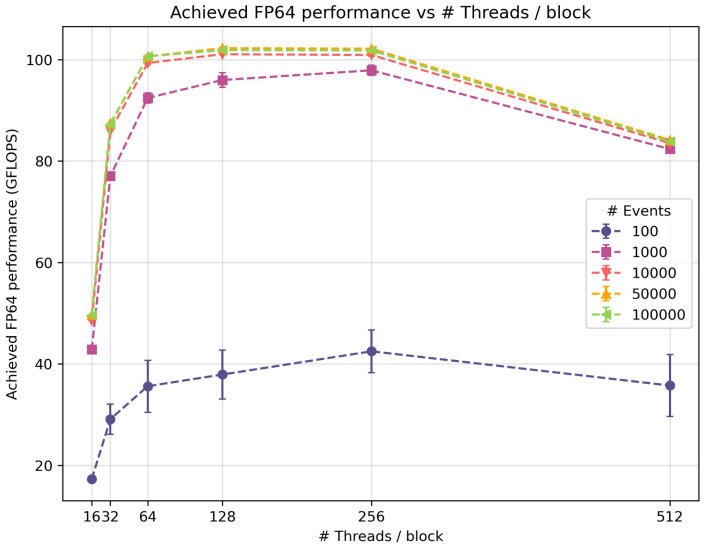
number of double precision FP addition/multiplication/FMA instructions actually executed	`smsp__sass_thread_inst_executed_op_dadd_pred_on.sum`
	`smsp__sass_thread_inst_executed_op_dmul_pred_on.sum`
	`smsp__sass_thread_inst_executed_op_dfma_pred_on.sum`
		Operations per event ~ 20 M

03

Benchmarking



theoretical BW: ~ 320 GB/s



reaching less than half of FP64 peak GPU performance

Recap & Comments

- Comparison between different Anti-k_T code implementations
- Benchmarks
- Comparison between execution times between different code implementations (best configuration for each implementation, whole dataset):

serial	**~ 10 min** with compiler O1-O3 optimization
OpenMP	**< 8 min** with 4 threads, dynamic schedule
CUDA	**~ 20 s** with 128 threads/block

30 times faster

Thank you for your attention!

GitHub repo

02

NVIDIA Tesla T4 GPU

Table 2. Memory Specifications

Specification	Description
Maximum memory clock	5001 MHz
Memory size	16 GB
Memory bus width	256 bits
Peak Memory bandwidth	Up to 320 GBytes/s

Note: The TU102 GPU also features 144 FP64 units (two per SM), which are not depicted in this diagram. The FP64 TFLOP rate is 1/32nd the TFLOP rate of FP32 operations. The small number of FP64 hardware units are included to ensure any programs with FP64 code operates correctly.

Figure 2. Turing TU102 Full GPU with 72 SM Units

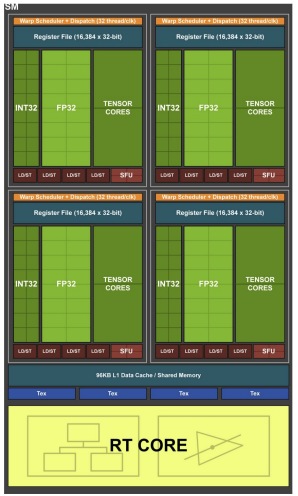


Figure 4. Turing TU102/TU104/TU106 Streaming Multiprocessor (SM)

Table 5. Comparison of the Pascal Tesla P4 and the Turing Tesla T4

GPU	Tesla P4 (Pascal)	Tesla T4 (Turing)
GPCs	4	5
TPCs	20	20
SMs	20	40
CUDA Cores/SM	128	64
CUDA Cores/GPU	2,560	2,560
Tensor Cores/SM	NA	8
Tensor Cores/GPU	NA	320
RT Cores	NA	40
GPU Base Clock MHz	810	585*
GPU Boost Clock MHz	1,063	1,590
Peak FP32 TFLOPS	5.5	8.1
Peak INT32 TIPS	NA	8.1
Peak FP16 TFLOPS	NA	16.2
Peak FP16 Tensor TFLOPS with FP16 Accumulate*	NA	65
Peak FP16 Tensor TFLOPS with FP32 Accumulate*	NA	65
Peak INT8 Tensor TOPS*	22	130
Peak INT4 Tensor TOPS*	NA	260
Frame Buffer Memory Size and Type	8192 MB GDDR5X	16384 MB GDDR6
Memory Interface	256-bit	256-bit
Memory Clock (Data Rate)	6 Gbps	10 Gbps
Memory Bandwidth (GB/sec)	192	320
ROPs	64	64
TDP	75 Watts	70 Watts

2x L1 Bandwidth
Lower L1 Hit Latency
Up to 2.7x L1 Capacity
2x L2 Capacity

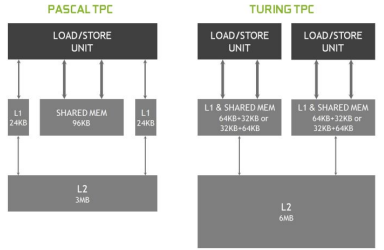


Figure 6. New Shared Memory Architecture

02

Parallel implementation on GPU

type	memory	memory access	variable scope	variable lifetime	memory speed	kernel executed on	kernel callable from	note
int var	register	thread	thread	kernel	high, at most 1 CC	-	-	limited memory
int var[len]	local	thread	thread	kernel	mid, ~10 CC	-	-	supports registers
device shared int	shared	threads in a block	block	kernel	on-chip, 10-100 CC	device	device	threads can collaborate
device int	global	all threads	grid	application	slow, 400-800 CC	device	device	data copied to/from CPU
device constant int	constant	all threads, but they can only read	grid	application	mid	device	device	optimised for broadcast access